

## Evolution of Attention in NLP (Step-by-Step Notes)

### 1] Basic Encoder–Decoder Architecture

#### Idea

We start with the classic **sequence-to-sequence (Seq2Seq)** model using RNNs/LSTMs.

- **Encoder** reads input sentence → converts into hidden states
- Final hidden state = **context vector (C)**
- **Decoder** uses this single vector to generate output

#### Problem (Very Important)

- Entire input sentence is compressed into **ONE vector**
- Works fine for short sentences
- Fails for long sentences → **information bottleneck**

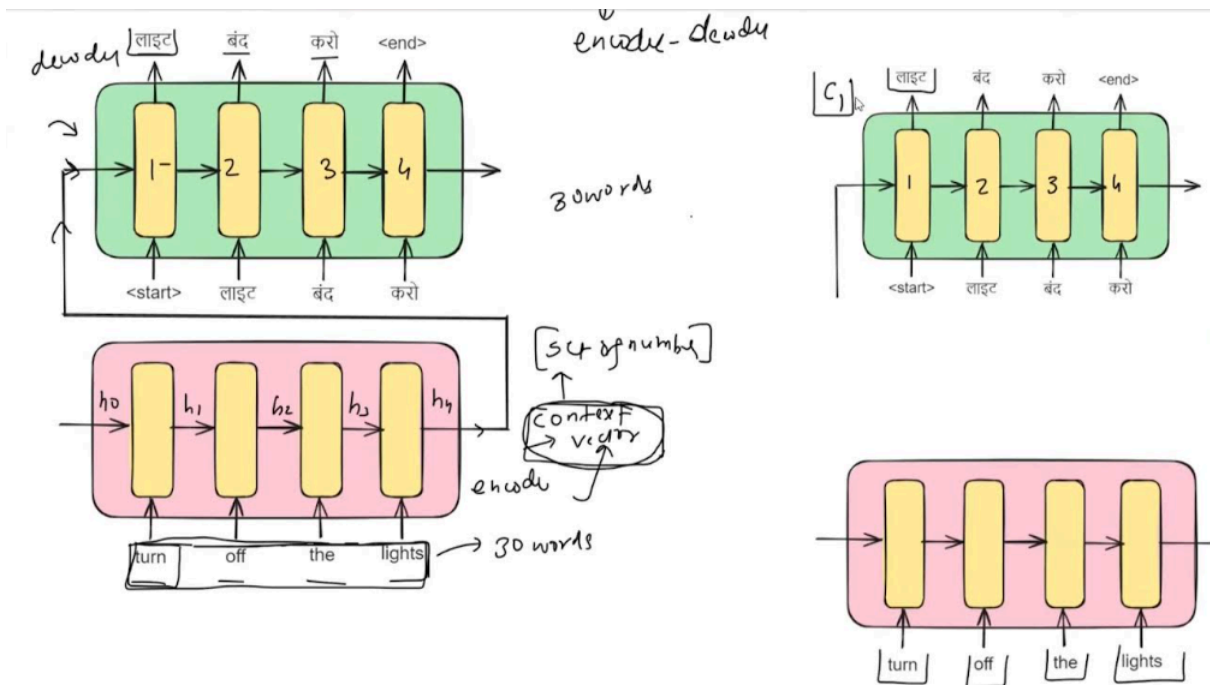
Example:

“The book that I bought yesterday from the store...”

The model struggles to remember early words when generating later outputs.

#### Key Limitation

- Fixed-length representation
- Loss of long-range dependencies
- Accuracy drops as sentence length increases



## 2 Introduction of Attention Mechanism

### Core Idea

Instead of using **one fixed context vector**, let the decoder:

“Look back” at **all encoder hidden states** when generating each word

### What Changes?

Instead of:

- One context vector for all steps

We now have:

- Different context vector for each decoder steps

### How it Works (High-Level)

For each decoder step:

1. Compare current decoder state with all encoder states
2. Compute importance scores (alignment scores)

3. Convert scores  $\rightarrow$  probabilities (softmax)  $\rightarrow \alpha$  (alpha weights)
4. Weighted sum of encoder states  $\rightarrow$  **context vector**

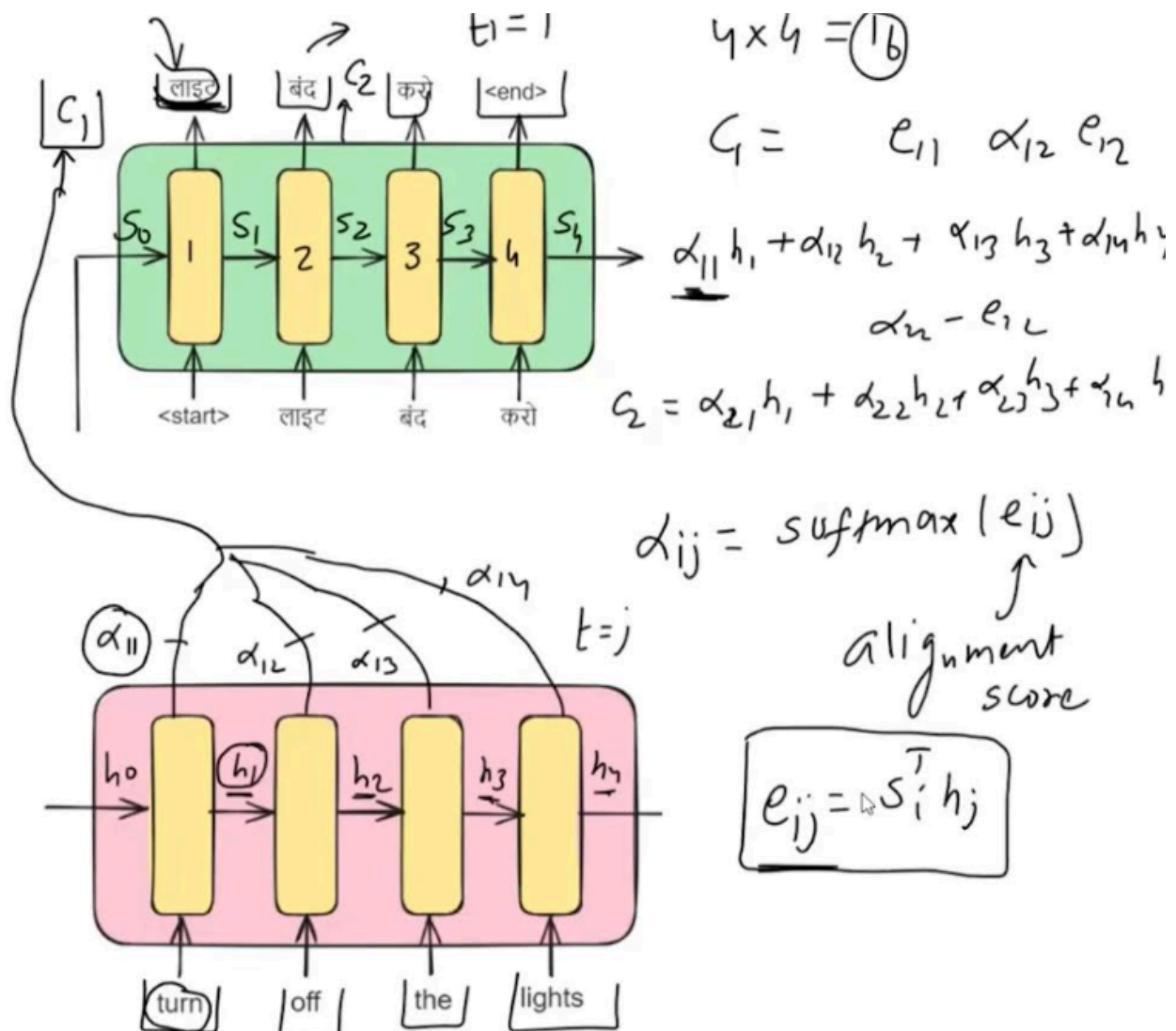
**Why is it called “Attention”?**

Because the model:

**“attends” to important words in the input**

Example (translation):

When translating “eating”, model focuses on the word “khana” instead of entire sentence.



### 3 Bahdanau Attention (Additive Attention)

## Introduced by

Dzmitry Bahdanau



## Key Idea

Use a **small neural network** to compute alignment scores

## Formula (Conceptual)

Score depends on:

- Decoder hidden state
- Encoder hidden state

Instead of simple similarity, it **learns** how to compare

## Steps

1. Combine encoder + decoder states
2. Pass through feedforward NN
3. Get score
4. Apply softmax  $\rightarrow \alpha$  weights
5. Compute context vector

## Advantages

- More flexible
- Learns better alignment

## Drawback

- Slower (more computation)

## ④ Luong Attention (Multiplicative / Dot-Product)

## Introduced by

Minh-Thang Luong

## Key Idea

Use **dot product** for similarity

## Formula (Conceptual)

score=decoder\_state · encoder\_state

## Variants

- Dot product
- General (with weight matrix)
- Scaled dot-product (later used in Transformers)

## Advantages

- Faster than Bahdanau
- Simpler computation

## Drawback

- Less expressive than additive attention

$$\left[ \begin{array}{l} c_i = \sum \alpha_{ij} h_j \\ \alpha_{ij} = \text{softmax}(e_{ij}) \\ e_{ij} = s_i^T h_j \end{array} \right] \rightarrow \text{Luong attention}$$

Handwritten notes:   
 - An arrow points to  $\alpha_{ij}$  with the label "weights".   
 - An arrow points to  $e_{ij}$  with the label "alignment score".

## Bahdanau vs Luong (Quick Comparison)

| Feature     | Bahdanau       | Luong       |
|-------------|----------------|-------------|
| Method      | Neural network | Dot product |
| Speed       | Slower         | Faster      |
| Flexibility | Higher         | Lower       |
| Complexity  | High           | Low         |

## 5 Transition to Self-Attention

### Problem with Previous Attention

- Still relies on:
  - Encoder → Decoder interaction
- Sequential processing (RNNs)
- Hard to parallelize

## 6 Self-Attention Mechanism

### Core Idea

Each word:

“looks at ALL other words in the SAME sentence”

### Why “Self” Attention?

Because:

- Input attends to **itself**
- No separate encoder-decoder interaction needed

Same sequence used for:

- Query
- Key
- Value

## 7 Query, Key, Value Intuition

For each word:

- **Query (Q)** → What am I looking for?
- **Key (K)** → What do I contain?
- **Value (V)** → What information I provide?

### Process

1. Compare Query with all Keys → similarity scores
2. Apply softmax → attention weights

3. Multiply weights with Values
4. Sum  $\rightarrow$  output vector

### Core Self-Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

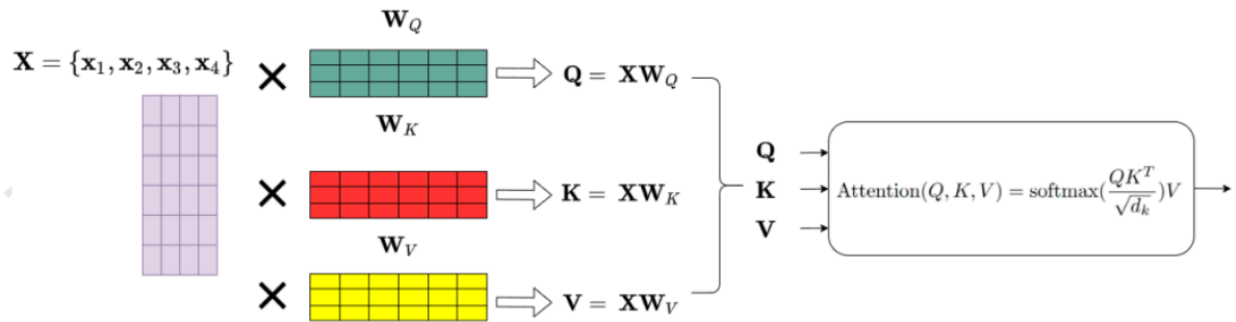
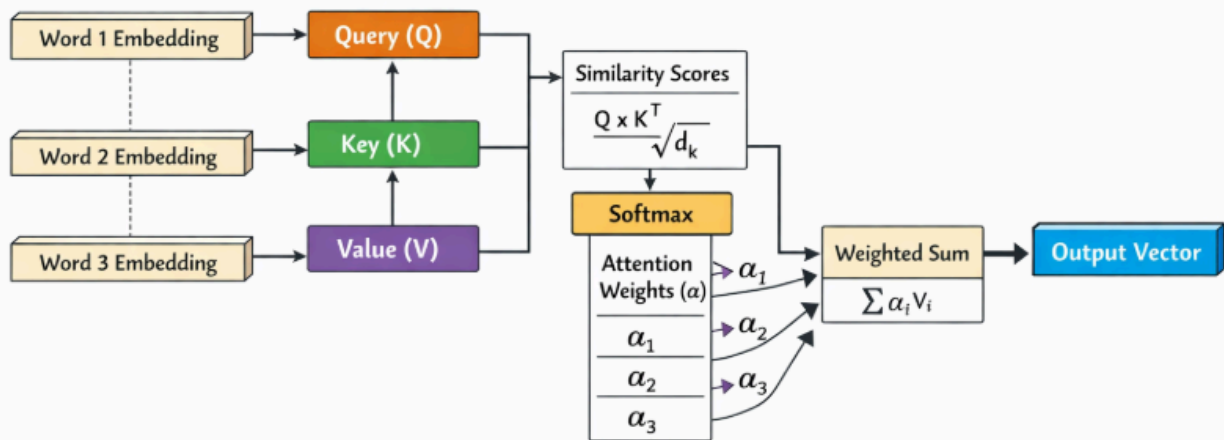
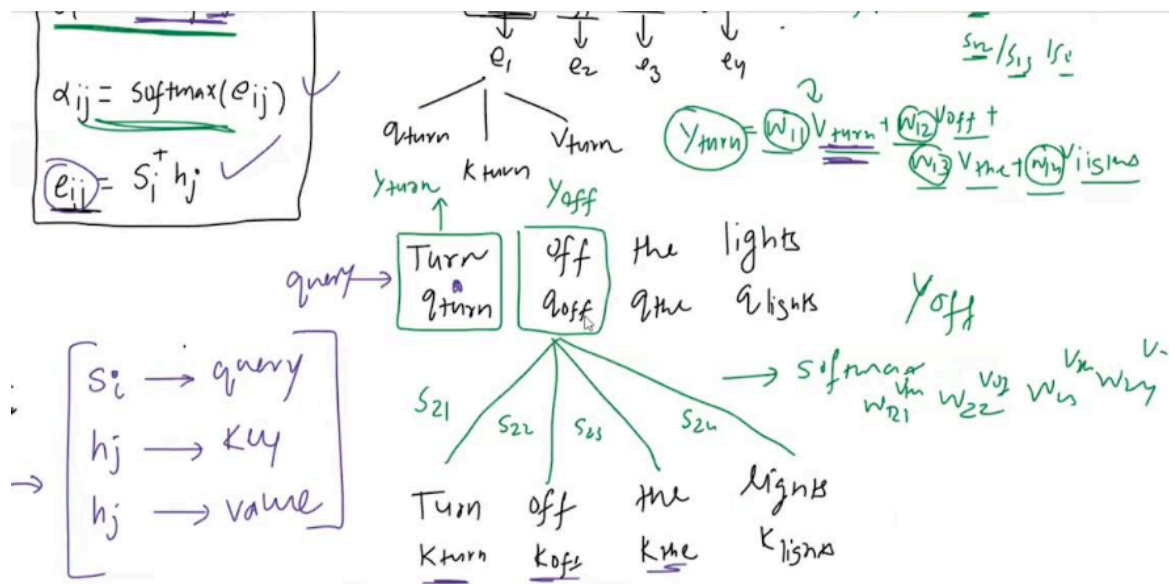
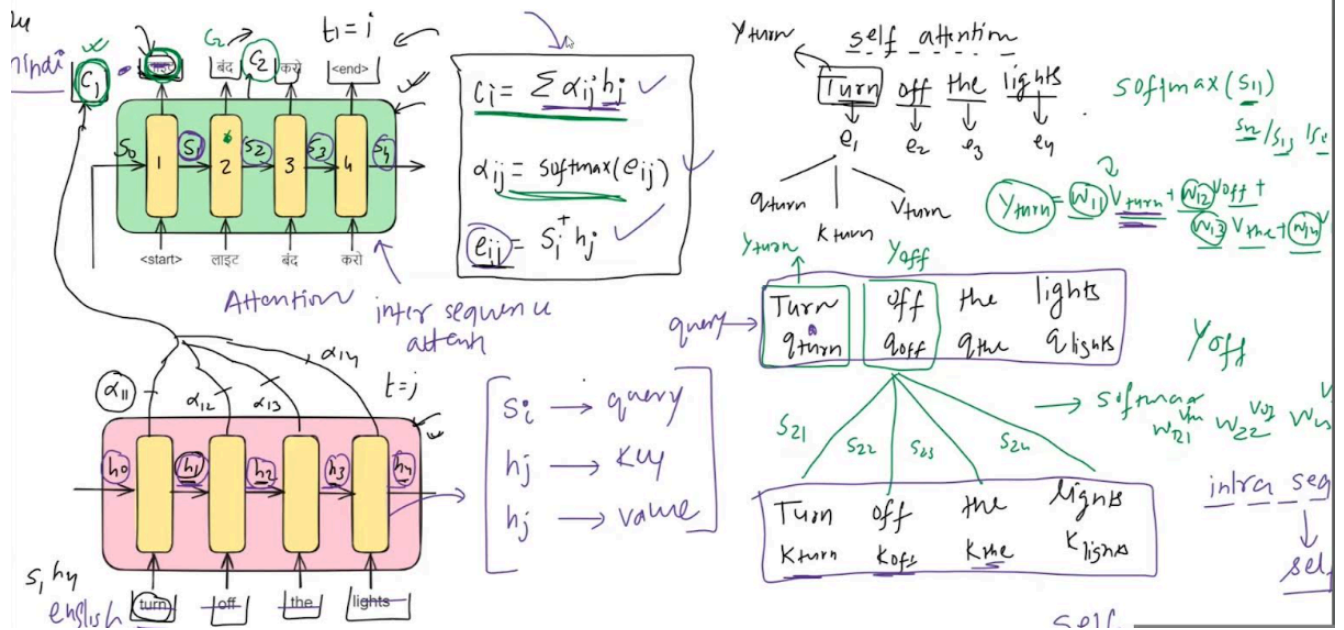


Figure 12: Key-query-value  
[adaloglou2021transformer]

## Self-Attention Mechanism





## 8 Why Self-Attention is Powerful

### Advantages

- Captures long-range dependencies
- Fully parallelizable
- No information bottleneck
- Dynamic relationships between words



## 9 Similarity: Luong Attention vs Self-Attention

### Key Insight

Self-attention is basically:

**Generalized Luong Attention applied within the same sequence**

### Similarities

| Concept                   | Luong Attention | Self-Attention |
|---------------------------|-----------------|----------------|
| Uses dot product          | ✓               | ✓              |
| Computes alignment scores | ✓               | ✓              |
| Uses softmax weights      | ✓               | ✓              |
| Weighted sum output       | ✓               | ✓              |

### Key Difference

| Feature           | Luong             | Self-Attention |
|-------------------|-------------------|----------------|
| Between sequences | Encoder → Decoder | Same sequence  |
| Q, K, V           | Not explicit      | Explicit       |
| Parallelism       | Limited           | High           |

## 10 Intuition: “Dynamic Movement in Vector Space”

Your explanation is GOOD—let’s refine it:

- Words are vectors in space
- Self-attention:
  - Pulls **relevant words closer**
  - Pushes irrelevant words away
- Final representation becomes **context-aware**

Example:

“bank”:

- Near “money” → financial meaning
- Near “river” → geographical meaning

### **Final Summary (Exam Ready)**

1. Encoder-decoder used a **single context vector** → bottleneck problem
2. Attention solved it by giving **dynamic context per output step**
3. Bahdanau → learned alignment (neural network)
4. Luong → dot-product alignment (faster)
5. Self-attention:
  - Uses same sequence
  - Introduces Q, K, V
  - Fully parallel & powerful
6. Self-attention  $\approx$  **Luong attention generalized + applied internally**